

在 Cloud Run 部署 LangChain 代理

來源：<https://codelabs.developers.google.com/deploy-lanchain-agent-on-cloud-run?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將建構 LangChain 代理程式應用程式，並部署至 Cloud Run。

目錄

1. [總覽](#)
2. [事前準備](#)
3. [建立 LangChain 代理程式](#)
4. [測試代理](#)
5. [部署至 Cloud Run](#)
6. [測試部署作業](#)
7. [清除所用資源](#)
8. [恭喜](#)

1. 總覽

代理是一種自主程式，會與 AI 模型對話，並使用手邊的工具和情境資訊執行目標導向的作業，還能根據事實自主決策！

我們使用 Agent Development Kit(ADK)、LangChain、smolagents 等代理框架建立代理。透過這類架構建立的代理程式應用程式可部署至 Cloud Run，並以無伺服器應用程式的形式提供給使用者。

在本程式碼研究室中，我們將使用 LangChain 建構代理，並將其部署至 Cloud Run。

建構項目

準備好從原型提示詞轉移到建構代理程式了嗎？我們將使用 LangChain 建立代理，以結構化格式取得歷史人物的相關資訊。本實驗室的學習內容包括：

1. 使用 LangChain 建構簡單的代理程式，以結構化格式生成歷史人物的相關資訊
2. 在本機執行代理程式，並確認代理程式運作正常
3. 將代理程式部署至 Cloud Run，並使用 Cloud Run 網址叫用

需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
 - 已啟用計費功能的 Google Cloud 專案。
-

2. 事前準備

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 按一下這個[連結](#)，啟動 Cloud Shell。如要在 **Cloud Shell 終端機** (用於執行雲端指令) 和編輯器 (用於建構專案) 之間切換，請點選 Cloud Shell 中的對應按鈕。
4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

□□□

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 請務必使用 Python 3.13 以上版本

如需其他 gcloud 指令和用法，請參閱[說明文件](#)。

步驟 3 · 約 0 分鐘

3. 建立 LangChain 代理程式

專案結構

在 Cloud Shell 中建立名為 `langchain-app` 的資料夾，並在其中新增下列檔案：

```
langchain-gemini-fastapi-app/  
|—— main.py  
|—— requirements.txt
```

應用程式代碼

您應在 `requirements.txt` 中新增下列依附元件：

```
fastapi  
uvicorn  
langchain  
langchain-google-genai  
python-dotenv
```

在 `main.py` 中，我們會編寫使用 Gemini 模型的代理程式碼，並擷取有關歷史人物的資訊。擷取後，按照指示將資料格式化為結構化格式。

這項實作作業使用新式 LCEL (LangChain 運算式語言) 語法。

```

import os
import uvicorn
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import JsonOutputParser

# Initialize FastAPI
app = FastAPI(title="LangChain App for Historical Figures")

# 1. Setup Gemini Model
# We expect GOOGLE_API_KEY to be set in the environment variables
llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    temperature=0.7
)

# 2. Define the Prompt
prompt = ChatPromptTemplate.from_template("You are an expert Historian. For the historical personality {name}, you are able to accurately tell their birth date and birth country. Return the output in the JSON format containing name, birthDate, birthCountry. In case you are unable to retrieve birthDate or birthCountry, just have the unknown values as null. ensure the response is a valid json object only.")
output_parser = JsonOutputParser()

# Chain: Prompt -> Model -> Json Output Parser
chain = prompt | llm | output_parser

# 3. Define Request Model
class QueryRequest(BaseModel):
    name: str

# 4. Define Endpoint
@app.post("/chat")
async def chat(request: QueryRequest):
    try:
        response = await chain.ainvoke({"name": request.name})
        return response
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/")
def health_check():
    return {"status": "ok", "service": "LangChain-Gemini-FastAPI"}

```

步驟 4 · 約 0 分鐘

4. 測試代理

部署前，您可以在本機測試。

1. 在 [AI Studio](#) 中建立 Gemini API 金鑰。
2. 將 Gemini API 金鑰匯出為：

```
export GOOGLE_API_KEY="AIzaSy..."
```

3. 執行伺服器。

```
uvicorn main:app --port 8080 --host 0.0.0.0
```

4. 使用下列 `curl` 指令測試代理程式：

```
curl -X POST http://localhost:8080/chat \  
-H "Content-Type: application/json" \  
-d '{"name": "Abraham Lincoln"}'
```

確認您取得的 JSON 結構化輸出內容包含姓名、出生日期和出生國家/地區。

步驟 5 · 約 0 分鐘

5. 部署至 Cloud Run

我們會使用 `gcloud run deploy` 指令將代理程式應用程式部署至 Cloud Run。下列指令會使用 Cloud Build 建構容器，並在一個步驟中將容器部署至 Cloud Run。

將 `YOUR_API_KEY` 替換成實際的 Gemini API 金鑰。

```
gcloud run deploy gemini-fastapi-service \
  --source . \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --set-env-vars GOOGLE_API_KEY=<YOUR_API_KEY>
```

注意：在實際工作環境中，建議使用 **Secret Manager** 處理 API 金鑰，而非使用純環境變數。

部署完成後，終端機中應該會顯示端點，即可開始使用。

步驟 6 · 約 0 分鐘

6. 測試部署作業

使用 Cloud Run 端點，並執行 `curl`，確保獲得預期結果。

```
curl -X POST <CLOUD_RUN_ENDPOINT> \  
  -H "Content-Type: application/json" \  
  -d '{"name": "Abraham Lincoln"}'
```

您應該會得到與在本機執行的應用程式相同的結果。

步驟 7 · 約 0 分鐘

7. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
-

步驟 8 · 約 0 分鐘

8. 恭喜

恭喜！您已成功建立並與部署在 Cloud Run 的 LangChain 代理互動！
